

機械学習（第11回）

ロジスティック回帰分析

2 クラス分類問題

健常者・患者の検査値データ

個体 No	健常者・患者 y	検査値 1 x_1	検査値 2 x_2
1	健常者	50	15.5
2	健常者	69	18.4
3	健常者	93	26.4
4	健常者	76	22.9
5	健常者	88	18.6
6	患者	43	16.9
7	患者	56	21.6
8	患者	38	12.2
9	患者	21	16.0
10	患者	25	10.5

他の例：良品 or 不良品, 購入 or 非購入, 倒産 or 非倒産

クラス分類の手法

- 判別分析（マハラノビス距離）
- ロジスティック回帰分析（2クラス分類）
- CART（Classification and Regression Trees）
- SVM（Support Vector Machine）
- K近傍法
- ニューラルネットワーク

ロジスティック回帰モデル(1)

ロジスティック回帰モデルは、特性値 x の個体が $y=1$ もしくは、 $y=0$ である確率を表す。

先の検査値データにおいて、ここでは、患者（ $y=1$ ）、健常者（ $y=0$ ）とする。

$$Pr(y = 1|X = x) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^m \beta_j x_j))}$$

先の検査値データならば、 $m=2$, $x_1 = \text{検査値1}$, $x_2 = \text{検査値2}$

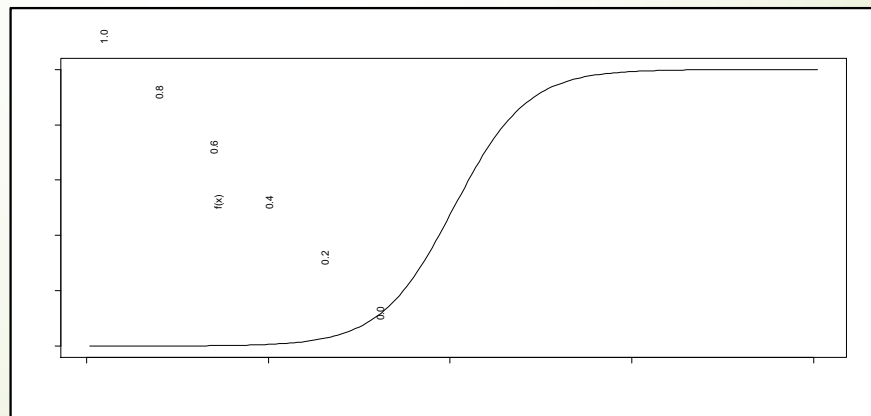
$$Pr(y = 1|X = x) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 x_1 + \beta_2 x_2))}$$

ロジスティック回帰モデル(2)

ロジスティック回帰モデルは、回帰直線 $z = \beta^T x$ を計算した後、
ロジスティック関数 $f(z) = \frac{1}{1+\exp(-z)}$ により0から1の値に変換する。

$$\begin{aligned} \mathbf{x} &= (1, x_1, x_2, \dots, x_m)^T \\ \boldsymbol{\beta} &= (\beta_0, \beta_1, \beta_2, \dots, \beta_m)^T \end{aligned}$$

ロジスティック関数のグラフ



ロジスティック関数は、 $(-\infty, +\infty)$ の値を
[0,1]の値に変換する。

パラメータ β_i の推定 (1)

個体 No	健常者・患者 y	検査値 1 x_1	検査値 2 x_2
1	健常者	50	15.5

個体No.1の場合の患者 ($y=1$) である確率

$$Pr(y = 1 | X = x) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 \times 50 + \beta_2 \times 15.5))}$$

a. $\beta_0 = 0.5, \beta_1 = -0.2, \beta_2 = 0.5$ とした場合

$$Pr(y = 1 | X = x) = 0.148$$

b. $\beta_0 = 1.0, \beta_1 = 0.1, \beta_2 = -0.2$ とした場合

$$Pr(y = 1 | X = x) = 0.948$$

a.の方が、実際の y の値 ($y=0$, 健常者) を反映している。

パラメータ β_i の推定 (2)

個体 i のクラスが $y=1$ である確率を $p(x_i; \beta)$ と表すと $y=0$ である確率は、 $1-p(x_i; \beta)$ と表せる。

実際のクラスが $y=1$ の場合 $\Rightarrow p(x_i; \beta)$ が大きい方が良い
実際のクラスが $y=0$ の場合 $\Rightarrow 1 - p(x_i; \beta)$ が大きい方が良い

2つの場合を
まとめた表現

$$p(x_i; \beta)^{y_i} (1 - p(x_i; \beta))^{1-y_i}$$

$$\begin{cases} y_i = 1 \Rightarrow p(x_i; \beta) \\ y_i = 0 \Rightarrow 1 - p(x_i; \beta) \end{cases}$$

パラメータ β_i の推定 (3) ~最尤推定~

個体間が独立（無関係）と仮定すると、 n 個の個体 $(x_i, y_i), i = 1, \dots, n$ に対する尤度は

$$\prod_{i=1}^n p(x_i; \beta)^{y_i} (1 - p(x_i; \beta))^{1-y_i}$$

$\prod_{i=1}^n a_i = a_1 \times a_2 \times \dots \times a_n$ を表す記号（ Σ の掛け算版）

積のままでは扱い難いため、対数をとった次の対数尤度

$$L(\beta) = \sum_{i=1}^n (y_i \log p(x_i; \beta) + (1 - y_i) \log(1 - p(x_i; \beta)))$$

を最大化する（クロスエントロピー誤差）。
対数尤度 $L(\beta)$ を最大にする様な β は、尤度も最大にする（ $\log x$ は単調増加関数）。

対数の性質
 $\log x^a y^b$
 $= a \log x + b \log y$

内積のベクトルによる微分 (復習)

$\mathbf{a}$ を定数ベクトルとすると内積 $\mathbf{x}^T \mathbf{a} (= \mathbf{a}^T \mathbf{x})$ のベクトル $\mathbf{x}$ による微分は次のようになる。

$$\begin{aligned} \frac{\partial \mathbf{x}^T \mathbf{a}}{\partial \mathbf{x}} &= \begin{pmatrix} \frac{\partial \mathbf{x}^T \mathbf{a}}{\partial x_1} \\ \frac{\partial \mathbf{x}^T \mathbf{a}}{\partial x_2} \\ \vdots \\ \frac{\partial \mathbf{x}^T \mathbf{a}}{\partial x_p} \end{pmatrix} = \begin{pmatrix} \frac{\partial \sum_{i=1}^p x_i a_i}{\partial x_1} \\ \frac{\partial \sum_{i=1}^p x_i a_i}{\partial x_2} \\ \vdots \\ \frac{\partial \sum_{i=1}^p x_i a_i}{\partial x_p} \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_p \end{pmatrix} \\ &= \mathbf{a} \end{aligned}$$

ロジスティック関数の微分

$p(\mathbf{x}_i; \boldsymbol{\beta}) = \frac{1}{1 + \exp(-\boldsymbol{\beta}^T \mathbf{x}_i)}$ を $\boldsymbol{\beta}$ で微分すると

ロジスティック関数 $p(z) = \frac{1}{1 + \exp(-z)}$ と $z = \boldsymbol{\beta}^T \mathbf{x}$

の合成関数であるから,

チェーンルール

$$\frac{\partial p}{\partial \boldsymbol{\beta}} = \frac{dp}{dz} \frac{dz}{\partial \boldsymbol{\beta}} = p(z)(1 - p(z))\mathbf{x}より,$$

$$\frac{\partial p}{\partial \boldsymbol{\beta}} = p(\mathbf{x}_i; \boldsymbol{\beta})(1 - p(\mathbf{x}_i; \boldsymbol{\beta}))\mathbf{x}_i$$

対数尤度の微分 (1)

対数尤度を最大にする β は、停留点であるので、微分を計算する.

$$\begin{aligned} L(\beta) &= \sum_{i=1}^n (y_i \log p(x_i; \beta) + (1 - y_i) \log(1 - p(x_i; \beta))) \\ \frac{\partial L(\beta)}{\partial \beta} &= \sum_{i=1}^n \left(\frac{y_i}{p(x_i; \beta)} p(x_i; \beta) (1 - p(x_i; \beta)) x_i \right. \\ &\quad \left. + \frac{1 - y_i}{1 - p(x_i; \beta)} (-p(x_i; \beta) (1 - p(x_i; \beta)) x_i) \right) \\ &= \sum_{i=1}^n (y_i (1 - p(x_i; \beta)) x_i - (1 - y_i) p(x_i; \beta) x_i) \\ &= \sum_{i=1}^n (y_i - p(x_i; \beta)) x_i \end{aligned}$$

チェーンルールと
対数の微分

$$(\log a)' = \frac{1}{a}$$

$$\begin{aligned} \frac{\partial p}{\partial \beta} &= p(x_i; \beta) \\ &\quad (1 - p(x_i; \beta)) x_i \end{aligned}$$

対数尤度の微分 (2)

$$L(\beta) = \sum_{i=1}^n (y_i \log p(x_i; \beta) + (1 - y_i) \log(1 - p(x_i; \beta)))$$

を最大化するには

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^n x_i (y_i - p(x_i; \beta)) = 0$$

となる β を求めれば良いが、解析的に β を解くことはできない

($\beta = \bigcirc \bigcirc \bigcirc$ と陽に β を表すことが出来ない)

⇒ ニュートン法による繰り返し計算

(復習) 最急降下法

関数 $f(x, y)$ の最小化

$$\begin{aligned} \text{更新則 } x_{i+1} &= x_i - \alpha \frac{\partial f}{\partial x}(x_i, y_i) \\ y_{i+1} &= y_i - \alpha \frac{\partial f}{\partial y}(x_i, y_i) \end{aligned}$$

ベクトル表記すると

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}, \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f}{\partial x}(x_i, y_i) \\ \frac{\partial f}{\partial y}(x_i, y_i) \end{pmatrix}$$

最急降下法の更新則は、次の様になる。

$$\mathbf{x}^{new} = \mathbf{x}^{old} - \alpha \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$$

ニュートン法

二階微分情報を使った関数 $f(x, y)$ の最小化

ヘッセ行列 (多変数関数の2階微分の行列)

$$\frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}} = \begin{pmatrix} \frac{\partial^2 f}{\partial x \partial x} & \frac{\partial^2 f}{\partial y \partial x} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y \partial y} \end{pmatrix}$$

ニュートン法の更新則

$$\mathbf{x}^{new} = \mathbf{x}^{old} - \left(\frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}} \right)^{-1} \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}$$

A^{-1} は行列
 A の逆行列
を表す

ニュートン法は収束が早い。

ヘッセ行列と勾配ベクトル (1階微分) を求める必要がある。

ロジスティック回帰のニュートン法による β の推定

$$\beta^{new} = \beta^{old} - \left(\frac{\partial^2 L(\beta)}{\partial \beta \partial \beta^T} \right)^{-1} \frac{\partial L(\beta)}{\partial \beta}$$

ヘッセ行列

勾配ベクトル

$$\frac{\partial^2 L}{\partial \beta \partial \beta^T} = \begin{pmatrix} \frac{\partial^2 L}{\partial \beta_0 \partial \beta_0} & \frac{\partial^2 L}{\partial \beta_0 \partial \beta_1} & \cdots & \frac{\partial^2 L}{\partial \beta_0 \partial \beta_m} \\ \frac{\partial^2 L}{\partial \beta_1 \partial \beta_0} & \frac{\partial^2 L}{\partial \beta_1 \partial \beta_1} & \cdots & \frac{\partial^2 L}{\partial \beta_1 \partial \beta_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 L}{\partial \beta_m \partial \beta_0} & \frac{\partial^2 L}{\partial \beta_m \partial \beta_1} & \cdots & \frac{\partial^2 L}{\partial \beta_m \partial \beta_m} \end{pmatrix} \quad \frac{\partial L}{\partial \beta} = \begin{pmatrix} \frac{\partial L}{\partial \beta_0} \\ \frac{\partial L}{\partial \beta_1} \\ \vdots \\ \frac{\partial L}{\partial \beta_m} \end{pmatrix}$$

勾配ベクトルとその行列表現

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^n x_i (y_i - p(x_i; \beta))$$

行列表現

$$\frac{\partial L}{\partial \beta} = (x_1 \ x_2 \ \cdots \ x_n) \begin{pmatrix} y_1 - p(x_1; \beta) \\ y_2 - p(x_2; \beta) \\ \vdots \\ y_n - p(x_n; \beta) \end{pmatrix}$$

$$X^T = (x_1 \ x_2 \ \cdots \ x_n), y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, p = \begin{pmatrix} p(x_1; \beta) \\ p(x_2; \beta) \\ \vdots \\ p(x_n; \beta) \end{pmatrix}$$

とすると, $\frac{\partial L}{\partial \beta} = X^T (y - p)$ と表せる.

ここで行列 X^T は, 行に変数, 列に個体 (サンプル) を配置しており, 行に個体, 列に変数を配置した X を転置した行列である.

対数尤度の2階微分

$$\frac{\partial L}{\partial \boldsymbol{\beta}} = \sum_{i=1}^n (y_i - p(\mathbf{x}_i; \boldsymbol{\beta})) \mathbf{x}_i = \sum_{i=1}^n y_i \mathbf{x}_i - \sum_{i=1}^n p(\mathbf{x}_i; \boldsymbol{\beta}) \mathbf{x}_i$$

をさらに $\boldsymbol{\beta}$ で微分する.

$$\begin{aligned} \frac{\partial L}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} &= - \sum_{i=1}^n \mathbf{x}_i p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta})) \mathbf{x}_i^T \\ &= - \sum_{i=1}^n \mathbf{x}_i \underbrace{p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta}))}_{\text{スカラー (単一の値)}} \mathbf{x}_i^T \end{aligned}$$

$$\begin{aligned} \frac{\partial p}{\partial \boldsymbol{\beta}} &= p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta})) \mathbf{x}_i \\ \frac{\partial p}{\partial \boldsymbol{\beta}^T} &= p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta})) \mathbf{x}_i^T \end{aligned}$$

$$\mathbf{x}_i = \begin{pmatrix} 1 \\ x_{i1} \\ x_{i2} \\ \vdots \\ x_{im} \end{pmatrix}, \quad \mathbf{x}_i^T = (1, x_{i1}, x_{i2}, \dots, x_{im})$$

ヘッセ行列とその行列表現

$$\frac{\partial^2 L(\boldsymbol{\beta})}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} = - \sum_{i=1}^n \mathbf{x}_i p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta})) \mathbf{x}_i^T$$

ヘッセ行列の行列表現

$$\frac{\partial^2 L(\boldsymbol{\beta})}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} = \mathbf{X} \mathbf{W} \mathbf{X}^T$$

$\mathbf{W}$: 対角要素が W_{ii} 非対角要素が0の対角行列

$$W_{ii} = p(\mathbf{x}_i; \boldsymbol{\beta}) (1 - p(\mathbf{x}_i; \boldsymbol{\beta}))$$

$\mathbf{X}$: $n \times (m + 1)$ のデータ行列, 1列目には1が格納される (β_0 に対応する)

$\mathbf{X}^T$: $\mathbf{X}$ の転置行列.

ニュートン法（行列表現）

更新則

1. $\boldsymbol{\beta}^{new} = \boldsymbol{\beta}^{old} - \left(\frac{\partial^2 L(\boldsymbol{\beta})}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} \right)^{-1} \frac{\partial L}{\partial \boldsymbol{\beta}}$
 $= \boldsymbol{\beta}^{old} + (\mathbf{XW}\mathbf{X}^T)^{-1} \mathbf{X}^T (\mathbf{y} - \mathbf{p})$
2. $\boldsymbol{\beta}^{new}$ のもとで $\mathbf{W}$ と $\mathbf{p}$ を更新する

cf.反復重み付き最小二乗法

(Iterative Reweighted Least Squares Method)

最小二乗法

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

個々のサンプルに対する重みを考慮し、
重み行列 $\mathbf{W}$ を用いて、次の式により、計算する。

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

回帰分析は誤差分散が均一と仮定するが、
不均一の場合等に、反復最小二乗法が用いられる。

β の推定のIRLSとしての解釈

$$\begin{aligned}\beta^{new} &= \beta^{old} + (XWX^T)^{-1}X^T(y - p) \\ &= (X^TWX)^{-1}X^TW(X\beta^{old} + W^{-1}(y - p)) \\ &= (X^TWX)^{-1}X^TWz \\ z &= X\beta^{old} + W^{-1}(y - p)\end{aligned}$$

ロジスティック回帰の β の推定は, 対角成分を

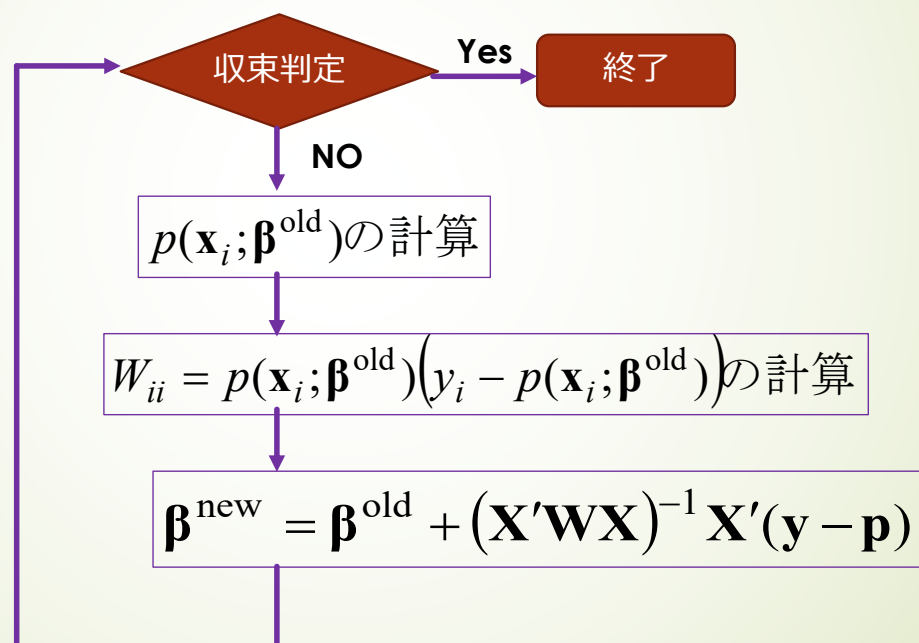
$$W_{ii} = p(x_i; \beta)(1 - p(x_i; \beta))$$

とした重み行列, 目的変数を

$$z = X\beta^{old} + W^{-1}(y - p)$$

とした反復重み付き最小二乗法と解釈できる.

ロジスティック回帰を プログラミングしてみよう！



ロジスティック回帰モデル

モデル構築（学習）フェーズ

訓練用データ $(\mathbf{x}_i, y_i), i = 1, \dots, n$
を用いて β を推定

$$\Pr(y = 1 \mid X = x) = \frac{1}{1 + \exp(-\beta' \mathbf{x})}$$

予測フェーズ

未知のデータ $(\mathbf{x}_{test}, y_{test})$
を予測

$$\Pr(y = 1 \mid \mathbf{x}_{test}) > 0.5 \Rightarrow y_{test} = 1$$

$$\Pr(y = 1 \mid \mathbf{x}_{test}) \leq 0.5 \Rightarrow y_{test} = 0$$

課題

- 表1の健常者・患者の検査値データを分類するロジスティック回帰モデルを構築するために、ニュートン法を用いて、 $\beta_0, \beta_1, \beta_2$ の推定を行え。
- 推定した $\beta_0, \beta_1, \beta_2$ を用いて表1の健常者・患者の検査値データを分類せよ。（推定された β , 個体の値 x_i を

$$\Pr(y = 1 \mid X = x) = \frac{1}{1 + \exp(-\beta' \mathbf{x})}$$

に代入して0.5を超えているか判別するプログラムを作れば良い)

ヒント

- 繰り返し使う演算（行列と行列の積，逆行列演算，転置行列は，関数（サブルーチン）化しておく，プログラムがわかりやすい。
- 作った関数は，簡単なデータで動作確認すると，どこで問題が起きているか確認できる。
- $p(\mathbf{x}_i; \boldsymbol{\beta})$ が極端に0または，1に近づきすぎると，逆行列が計算できない問題が生じるので，ある閾値を超えたら，閾値の値の置き換えるなどの処理が必要である。

Pythonのコード（1）

#ロジスティック関数の値を計算する関数

```
def logit(x,beta):  
    p= 1/(1+np.exp(-np.dot(x, beta)))  
    for i in range(len(p)):  
        if p[i] == 1:  
            p[i] = 0.999999  
    return p
```

#対数尤度を計算する関数

```
def loglike(x,y,beta):  
    p = logit(x,beta)  
    return sum(y*np.log(p)+(1-y)*np.log(1-p))
```

#データセットと計算した $\boldsymbol{\beta}$ を用いてクラスの予測を行う関数

```
def pred_logit(x,beta):  
    yhat=logit(x,beta)>0.5  
    return yhat.astype(int)
```

Pythonのコード (2)

```
#IRLSにより、 $\beta$ の計算を行う関数
def IRLS(x,y):
    EPS = 0.01; beta = [0,0,0]; MAX_ITER = 25
    for i in range(MAX_ITER):
        ll = loglike(x,y,beta)
        print("Log likelihood:{}".format(ll))
        p = logit(x,beta)
        w = p * (1 - p)
        w = np.diag(w)
        A = np.dot(np.dot(x.T,w),x)
        A_inv = np.linalg.inv(A)
        d = np.dot(np.dot(A_inv,x.T),(y - p))
        beta = beta + d
        sad = sum(abs(d))
        #print("sad={}".format(sad))
        if sad < EPS:
            break
    return beta
```

Pythonのコード (3)

```
import numpy as np
#データの準備
#yはクラスラベル y=0 (健常者) y=1 (患者)
#x1:検査値1, x2:検査値2
y = np.concatenate([np.zeros(5),np.ones(5)])
x0 = np.ones(10)
x1 = [50,69,93,76,88,43,56,38,21,25]
x2 = [15.5,18.4,26.4,22.9,18.6,16.9,21.6,12.2,16,10.5]
x = np.array([x0,x1,x2]).T
# $\beta$ の計算
beta = IRLS(x,y)
print(beta)
#クラスラベルの予測
pred = pred_logit(x,beta)
print(pred)
```


計算結果

$$\hat{\beta}_0 = 81.77, \hat{\beta}_1 = -4.70, \hat{\beta}_2 = 9.03$$

個体 No	健常者・患者 y	予測結果
1	健常者	健常者
2	健常者	健常者
3	健常者	健常者
4	健常者	健常者
5	健常者	健常者
6	患者	患者
7	患者	患者
8	患者	患者
9	患者	患者
10	患者	患者

```
Log likelihood:-6.931471805599453
Log likelihood:-2.923769617936077
Log likelihood:-1.8090805292970082
Log likelihood:-1.1311475392928518
Log likelihood:-0.7073193579110145
Log likelihood:-0.33010496906628833
Log likelihood:-0.11496145991596626
Log likelihood:-0.04114662495013721
Log likelihood:-0.015009499767031838
Log likelihood:-0.005530463157237295
Log likelihood:-0.0020619269472912536
Log likelihood:-0.0007880005945734373
Log likelihood:-0.0003195006068733873
Log likelihood:-0.00014819919731220472
Log likelihood:-8.022140472785628e-05
Log likelihood:-4.99685134848611e-05
Log likelihood:-3.446420575759915e-05
Log likelihood:-2.5478680106779774e-05
Log likelihood:-1.9768123691386148e-05
Log likelihood:-1.5885987847163266e-05
Log likelihood:-1.3111612071708828e-05
Log likelihood:-1.1052408000559758e-05
Log likelihood:-9.478685789028197e-06
Log likelihood:-8.248080748137995e-06
Log likelihood:-7.268079864518608e-06
[81.77020184 -4.69686532  9.03110504]
[0 0 0 0 0 1 1 1 1 1]
```

sklearnによるロジスティック回帰分析の実行

```
import numpy as np
#LogisticRegression関数を用いる場合はx0は必要ないので、以下の様にxを作る
x = np.array([x1,x2]).T
```

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression() #ロジスティック回帰モデルのインスタンスを作成
lr.fit(x,y)#ロジスティック回帰モデルの重みを学習
```

▼ LogisticRegression

```
LogisticRegression()
```

```
print("coefficient=",lr.coef_)
print("intercept=",lr.intercept_)
Y_pred=lr.predict(x)
print(Y_pred)
```

```
coefficient= [[-0.45716019  0.80814989]]
intercept= [9.60090909]
[0. 0. 0. 0. 0. 1. 1. 1. 1. 1.]
```

$$\hat{\beta}_0 = 9.601,$$
$$\hat{\beta}_1 = -4.457,$$
$$\hat{\beta}_2 = 0.808$$

自作関数によるロジスティック回帰分析と結果が異なっている。
これは、sklearnのLogisticRegressionでは、対数尤度 + L2正則化ペナルティ項を最大化していることと、データが完全分離なデータであることに起因している。